2. Специальная часть

2.1. Реализация серверного приложения

Серверная часть функционирует как REST API, обмениваясь с мобильным клиентом данными в формате JSON. Безопасность эндпоинтов реализована на базе авторизации по протоколу OAuth2 с использованием Bearer JWT-токенов подписи HS256.

Для хэширования паролей на стороне бэкенда используется библиотека bcrypt, исключающая хранение паролей пользователей в чистом виде.

Часть разработанных API-эндпоинтов серверной части представлена на рисунках 2.1.1 – 2.1.2:

```python
@router.post("/register", response_model=UserResponse, status_code=status.HTTP_201_CREATED)
def register(user_data: UserCreate, db: Session = Depends(get_db)):
    # Проверяем, существует ли уже пользователь с таким email.
    db_user = db.query(User).filter(User.email == user_data.email).first()
    if db_user:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Пользователь с таким email уже зарегистрирован"
        )
      You, 3 часа назад • working
    # Создаем нового пользователя.
    hashed_pw = hash_password(user_data.password)
    new_user = User(
        email=user_data.email,
        hashed_password=hashed_pw,
        role="listener"  # По умолчанию регистрируем с ролью Слушателя.
    )
    db.add(new_user)
    db.commit()
    db.refresh(new_user)
    return new_user

@router.post("/login", response_model=Token)
def login(user_data: UserLogin, db: Session = Depends(get_db)):
    # Поиск пользователя.
    db_user = db.query(User).filter(User.email == user_data.email).first()
    if not db_user or not verify_password(user_data.password, db_user.hashed_password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Неверный email или пароль"
        )

    # Генерация JWT.
    access_token = create_access_token(data={"sub": db_user.email, "role": db_user.role})
    return Token(access_token=access_token, role=db_user.role)

@router.get("/users", response_model=List[UserResponse])
def list_users(
    db: Session = Depends(get_db),
    current_admin: User = Depends(get_current_admin)
):
    """Возвращает список всех зарегистрированных пользователей (только для Администратора)."""
    return db.query(User).all()
```

Рисунок 2.1.1 – Часть разработанных API-эндпоинтов серверной части

```python
# Публикация подкаста (только для роли `Администратора`).      You, 2 часа назад • comple
@router.post("/", response_model=PodcastResponse, status_code=status.HTTP_201_CREATED)
async def create_podcast(
    title: str = Form(...),
    category: str = Form(...),
    description: str = Form(None),
    file: UploadFile = File(...),
    db: Session = Depends(get_db),
    current_admin: User = Depends(get_current_admin)
):
    # Проверка расширения файла.
    if not file.filename.lower().endswith(".mp3"):
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Разрешены только файлы формата .mp3"
        )

    # Проверка MIME-типа.
    if file.content_type != "audio/mpeg":
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Неверный MIME-тип. Разрешен только audio/mpeg"
        )

    # Валидация размера файла (до 100 МБ).
    if file.size is not None:
        file_size = file.size
    else:
        # Резервный способ через прямое обращение к внутреннему файловому объекту.
        file.file.seek(0, 2)
        file_size = file.file.tell()
        file.file.seek(0)

    if file_size > 100 * 1024 * 1024:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail="Размер файла превышает лимит в 100 МБ"
        )
```

Рисунок 2.1.2 – Часть разработанных API-эндпоинтов серверной части

2.2. Разработка пользовательского интерфейса и логики

Интерфейс мобильного приложения разработан в соответствии с гайдлайнами Material Design 3 и ориентирован на темную цветовую палитру (Dark Premium) для снижения нагрузки на зрение при длительном использовании.

Проектирование функциональных возможностей мобильного приложения базируется на принципе разделения прав доступа между двумя категориями пользователей: Слушателем и Администратором. Для формализации требований к системе и описания сценариев взаимодействия пользователей с интерфейсом была разработана диаграмма прецедентов (Use Case Diagram).

Данная диаграмма позволяет на концептуальном уровне визуализировать иерархию функций приложения, выделив ключевые действия, доступные каждому из акторов. Сценарии использования включают в себя как базовые операции по работе с контентом (просмотр каталога, воспроизведение подкастов), так и административные функции по управлению базой данных и публикацией новых медиафайлов.

На рисунке 2.2.1 представлена Use Case диаграмма.

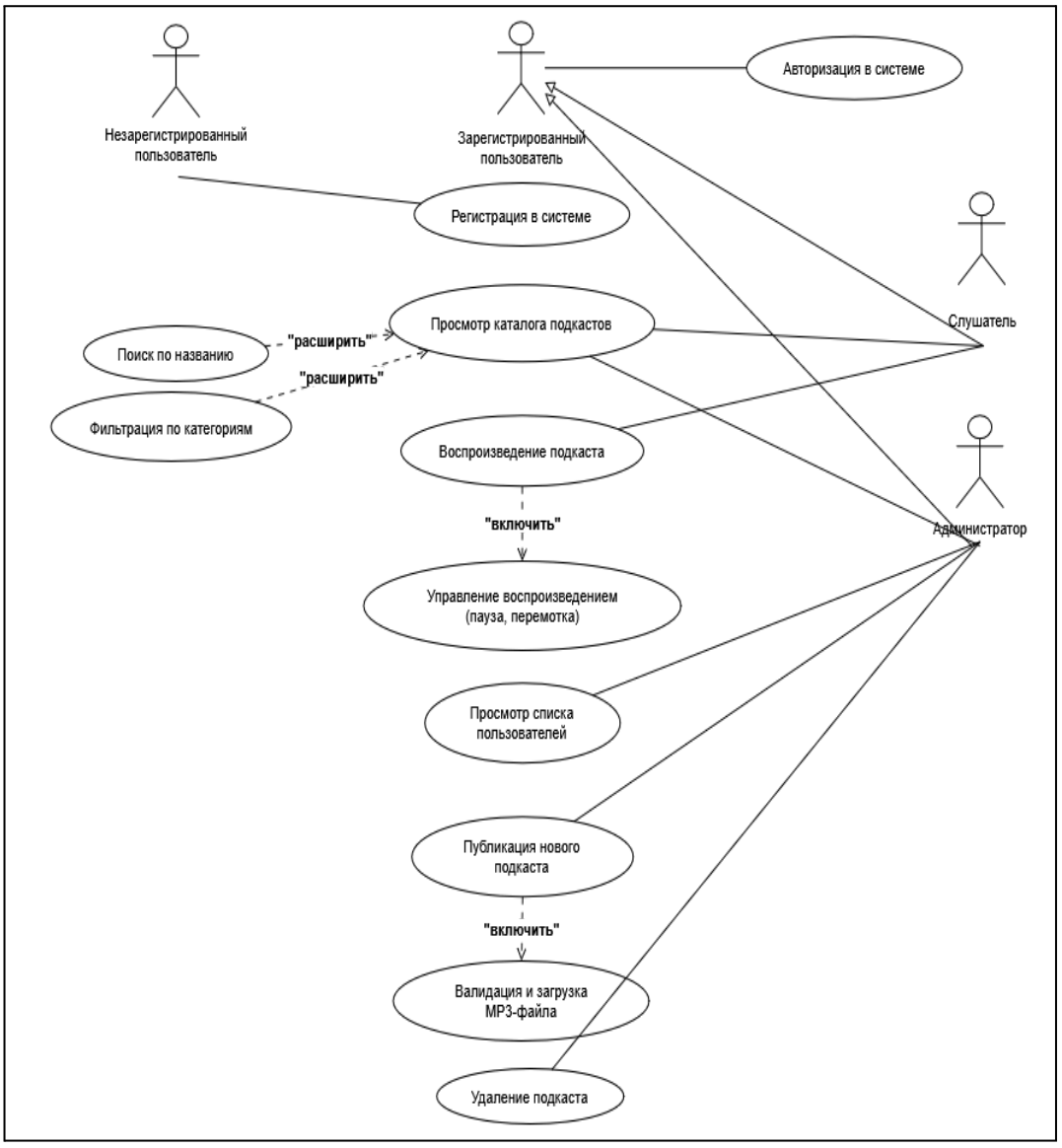


Рисунок 2.2.1 – Use Case диаграмма

Для детального проектирования алгоритмов обмена данными между клиентским приложением и серверной частью на основе выделенных прецедентов была разработана BPMN-диаграмма взаимодействия процессов.

На рисунке 2.2.2 представлена BPMN-диаграмма взаимодействия процессов.

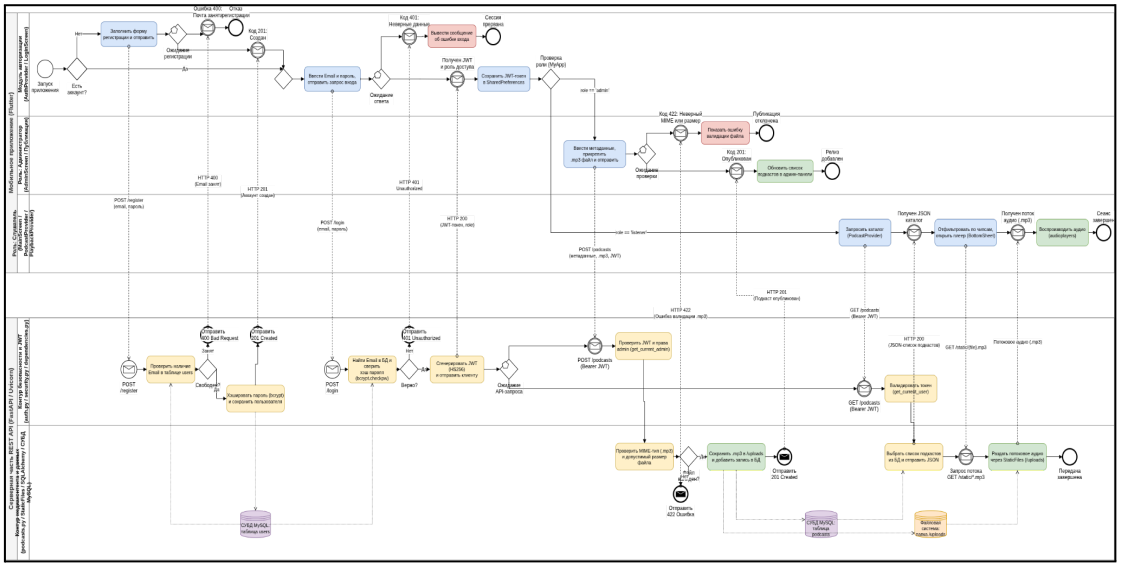


Рисунок 2.2.2 – BPMN-диаграмма взаимодействия процессов

					40.Ф-830-2026 09.02.07 КП-ПЗ		
Изм.	Лист	№ докум.	Подпись	Дата			
Разраб.		Файзельгаянов И.Ф.			Разработка мобильного приложения для прослушивания подкастов	Лит.	Лист 1 / Листов 44
Провер.		Фатхуллова О.В.					
						УКСИВТ 23П-4	

2.2.1. Архитектура клиентских модулей

Логика приложения разделена на три ключевых провайдера:

- модуль AuthProvider сохраняет JWT-токен во внутреннем хранилище устройства через shared_preferences и управляет сессией пользователя;
- модуль PodcastProvider запрашивает список подкастов у API сервера, обрабатывает контекстный поиск и горизонтальную фильтрацию по чипсам категорий;
- модуль PlaybackProvider инкапсулирует работу с нативным аудиоплеером смартфона посредством пакета audioplayers и отслеживает текущую позицию вещания, буферизацию, общую длительность трека и статус воспроизведения.

Интерфейс плеера (BottomSheet) - для экрана воспроизведения реализована интерактивная выдвижная панель. Чтобы избежать конфликтов переполнения разметки на этапе изменения размеров панели в момент анимации раскрытия (от 84px до 275px), используется адаптивный контейнер SingleChildScrollView.

2.3. Протокол тестирования разработанной информационной системы

В ходе приемочного тестирования системы были проверены все критические сценарии использования программы.

В таблицах 2.3.1 – 2.3.4 представлены протоколы тестирования.

Таблица 2.3.1 – Протокол тестирования успешной регистрации слушателя

Параметр	Описание
Дата теста	25.06.2026
Приоритет тестирования	Высокий
Заголовок/название теста	Тестирование регистрации при корректных данных
Резюме испытания	Необходимо добиться корректного поведения системы при регистрации нового слушателя
Этапы теста	Открыть форму регистрации; заполнить поля «Email», «Пароль», «Повторный пароль»; нажать кнопку «Зарегистрироваться»
Тестовые данные	Email: testingList@gmail.com, Пароль: 7878488Aa, Повторный пароль: 7878488Aa
Ожидаемый результат	Учётная запись создана
Фактический результат	Учётная запись создана. Результат представлен на рисунке 2.3.1



Рисунок 2.3.1 – Фактический результат

Таблица 2.3.2 – Протокол тестирования регистрации слушателя с уже занятой почтой

Параметр	Описание
1	2
Дата теста	25.06.2026
Приоритет тестирования	Высокий
Заголовок/название теста	Тестирование регистрации при некорректных данных
Резюме испытания	Необходимо добиться корректного поведения системы при попытки регистрации нового слушателя с уже существующей почтой
Этапы теста	Открыть форму регистрации; заполнить поля «Email (уже существующего слушателя)», «Пароль», «Повторный пароль»; нажать кнопку «Зарегистрироваться»

Продолжение таблицы 2.3.2

1	2
Тестовые данные	Email: testingList@gmail.com, Пароль: 7878488Aa, Повторный пароль: 7878488Aa
Ожидаемый результат	Ошибка, учетная запись не создана
Фактический результат	Ошибка, учетная запись не создана. Результат представлен на рисунке 2.3.2



Рисунок 2.3.2 – Фактический результат

Таблица 2.3.3 – Протокол тестирования успешной аутентификации

Параметр	Описание
1	2
Дата теста	25.06.2026
Приоритет тестирования	Высокий

Продолжение таблицы 2.3.3

1	2
Заголовок/название теста	Тестирование аутентификации при корректных данных
Резюме испытания	Необходимо добиться корректного поведения системы при аутентификации пользователя
Этапы теста	Открыть форму авторизации; заполнить поля «Email», «Пароль»; нажать кнопку «Авторизоваться»
Тестовые данные	Email: testingList@gmail.com, Пароль: 7878488Aa
Ожидаемый результат	Успешная аутентификация, получение JWT-токена
Фактический результат	Успешная аутентификация, получение JWT-токена. Результат представлен на рисунке 2.3.3

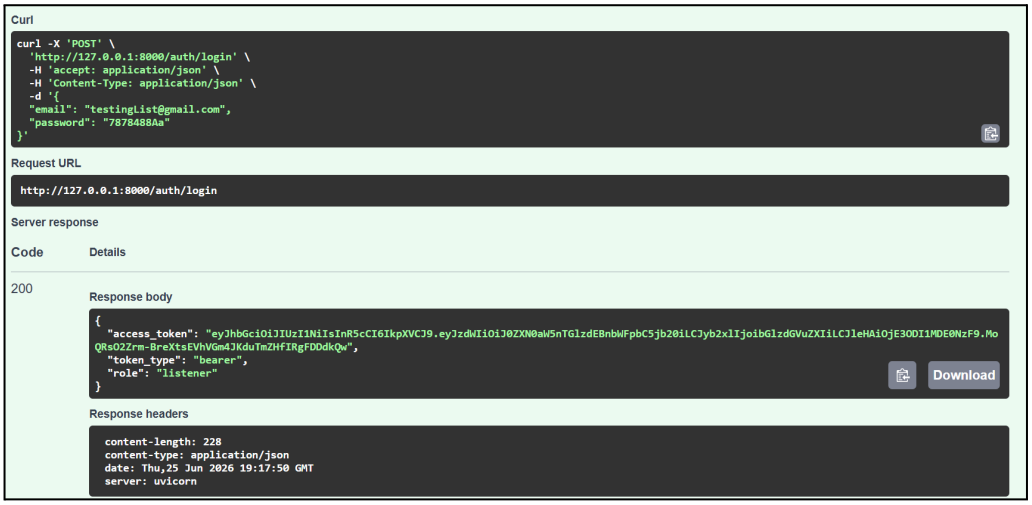


Рисунок 2.3.3 – Фактический результат

Таблица 2.3.4 – Протокол тестирования аутентификации с неверными данными

Параметр	Описание
Дата теста	25.06.2026
Приоритет тестирования	Высокий
Заголовок/название теста	Тестирование аутентификации при некорректных данных
Резюме испытания	Необходимо добиться корректного поведения системы при попытки аутентификации пользователя с некорректными данными
Этапы теста	Открыть форму авторизации; заполнить поля «Email», «Пароль»; нажать кнопку «Авторизоваться»
Тестовые данные	Email: aa@gsd, Пароль: -
Ожидаемый результат	Ошибка, неудачная аутентификация
Фактический результат	Ошибка, неудачная аутентификация. Результат представлен на рисунке 2.3.4

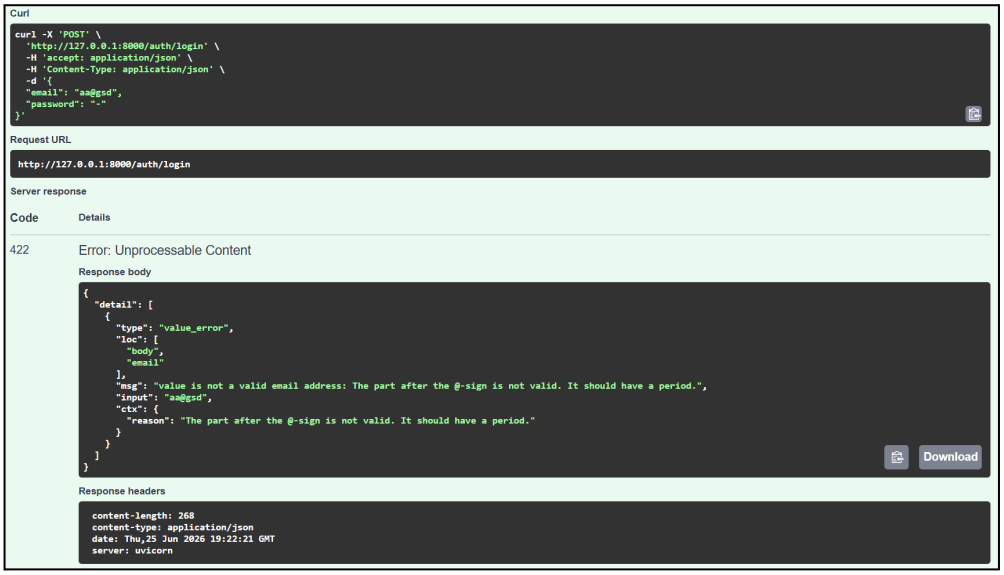


Рисунок 2.3.4 – Фактический результат